Vel Tech Group of Institutions

Name: Gontu Yaswanth
Email: vtu33434@veltech.edu.in
Roll no: Vtu33434
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 12_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement:

Raju has a hot dog of a certain length that he wishes to divide strategically to maximize the number of friends he can serve. The effectiveness of a given partition is determined by the product of its segment lengths. Raju's goal is to determine the optimal way to cut the hot dog such that this product is maximized, thereby maximizing the number of friends he can serve.

Example

Input:

4

Output:

Maximum friends served 4

Explanation:

For a hot dog of length 4, possible partitions and their corresponding products are:

1, 3 (1 × 3) = 3

2, 2 (2 × 2) = 4 (Maximum)

3, 1 (3 × 1) = 3

1, 1, 2 (1 × 1 × 2) = 2

1, 2, 1 (1 × 2 × 1) = 2

2, 1, 1 (2 × 1 × 1) = 2

1, 1, 1, 1 (1 × 1 × 1 × 1) = 1

Thus, the optimal partition results in a maximum product of 4, serving the highest number of friends.

Input Format

The input consists of a single integer n, representing the length of the hot dog.

Output Format

The output prints the maximum number of friends served.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Output: Maximum friends served 4

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return a > b ? a : b;
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    if (n == 1) {
        printf("Maximum friends served 1");
        return 0;
    }

    int dp[30];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = 0;
        for (int j = 1; j < i; j++) {
            dp[i] = max(dp[i], max(j * (i - j), j * dp[i - j]));
        }
    }

    printf("Maximum friends served %d", dp[n]);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Imagine you are given a set of items, each with a weight, a value, and a penalty value. You need to solve the Fractional Knapsack problem with the objective of maximizing the total value while minimizing penalties. Implement a greedy algorithm to choose items that best meet this objective.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 14.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 1: (7 - 1) / 2 = 3

Item 2: (12 - 4) / 3 = 2

Start filling the knapsack:

Add all of Item 1 (Weight: 2, Value: 7 - 1 = 6)

Add 1 unit of Item 2 (Weight: 3, Value: 12 - 4 = 8)

The total value is 6 + 8 = 14.

So, the correct output for the given input is 14.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 43.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 2: (30 - 5) / 10 = 2.5

Item 1: (20 - 2) / 5 = 3.6

Start filling the knapsack:

Add all of Item 1 (Weight: 5, Value: 20 - 2 = 18)

Add 1 unit of Item 2 (Weight: 10, Value: 30 - 5 = 25)

Now, let's calculate the total value of the knapsack:

Total Weight = 5 + 10 = 15 (fits within the knapsack's capacity)

Total Value = 18 + 25 = 43

Input Format

The first line contains an integer 'N' representing the number of items.

The next 'N' lines contain information for each item:

Three integers 'weight,' 'value,' and 'penalty,' are separated by spaces, where 'weight' is the weight of the item, 'value' is the value of the item, and 'penalty' is the penalty for taking the item.

The last line contains an integer 'knapsackWeight' representing the weight limit of the knapsack.

Output Format

The output prints "Maximum total value while minimizing penalties: " followed by the maximum total value achievable while minimizing penalties, rounded to two decimal places.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2
2 7 1
3 12 4
5

Output: Maximum total value while minimizing penalties: 14.00

Answer

```
// You are using GCC
#include <stdio.h>

struct Item {
    int w, v, p;
    double ratio;
};

void sort(struct Item a[], int n) {
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (a[i].ratio < a[j].ratio) {
                struct Item t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
}

int main() {
    int n;
    scanf("%d", &n);

    struct Item a[20];

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &a[i].w, &a[i].v, &a[i].p);
        a[i].ratio = (double)(a[i].v - a[i].p) / a[i].w;
    }

    int W;
    scanf("%d", &W);

    sort(a, n);
```

```
    double total = 0.0;

    for (int i = 0; i < n && W > 0; i++) {
        if (a[i].w <= W) {
            total += (a[i].v - a[i].p);
            W -= a[i].w;
        } else {
            total += (a[i].v - a[i].p) * ((double)W / a[i].w);
            W = 0;
        }
    }

    printf("Maximum total value while minimizing penalties: %.2f", total);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

There are n participants in a race. Each participant is assigned a performance score given in the integer array performance.

You are responsible for awarding medals to these participants, subject to the following conditions:

Each participant must receive at least one medal.

Participants with a higher performance score than their neighbors must receive more medals than their neighbors.

Return the minimum number of medals you need to distribute to the participants.

Example 1:

Input: ratings = [1,0,2]

Output: 5

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 1 medal (since their performance is the lowest).

Participant 3 gets 2 medals (since their performance is higher than Participant 2).

The total number of medals required is 5.

Example 2:

Input: ratings = [1,2,2]

Output: 4

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 2 medals (since their performance is better than Participant 1).

Participant 3 gets 1 medal (because their performance is equal to Participant 2, and they don’t need more medals).

The total number of medals required is 4.

Input Format

The first line consists of an integer n representing the number of participants.

The second line consists of n space-separated integers representing the performance scores of the participants.

Output Format

An integer representing the minimum number of medals required to satisfy the conditions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 0 2
Output: 5

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[20005], med[20005];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        med[i] = 1;
    }

    // Left to Right
    for (int i = 1; i < n; i++) {
        if (arr[i] > arr[i - 1])
            med[i] = med[i - 1] + 1;
    }

    // Right to Left
    for (int i = n - 2; i >= 0; i--) {
        if (arr[i] > arr[i + 1] && med[i] <= med[i + 1])
            med[i] = med[i + 1] + 1;
    }

    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += med[i];

    printf("%d", sum);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

You are planning a circular road trip that passes through several gas stations. Each station provides a certain amount of fuel, and traveling from one station to the next consumes a specific amount of fuel.

Your goal is to determine a starting gas station index from which you can begin your journey such that you complete the entire circuit in a clockwise direction without running out of fuel at any point.

Given two integer arrays:

gas[i] represents the fuel available at the i-th station,

cost[i] represents the fuel required to travel from station i to station i+1 (circularly),

Return the starting station index that allows completing the full trip.

If no such station exists, return -1.

Example 1

Input:

5

1 2 3 4 5

3 4 5 1 2

Output:

3

Explanation:

Start at station 3 (index 3) and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 4. Your tank = 4 - 1 + 5 = 8

Travel to station 0. Your tank = 8 - 2 + 1 = 7

Travel to station 1. Your tank = 7 - 3 + 2 = 6

Travel to station 2. Your tank = 6 - 4 + 3 = 5

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

Example 2

Input

3

2 3 4

3 4 3

Output:

-1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 0. Your tank = 4 - 3 + 2 = 3

Travel to station 1. Your tank = 3 - 3 + 3 = 3

You cannot travel back to station 2, as it requires 4 units of gas but you only have 3.

Therefore, you can't travel around the circuit once no matter where you start.

Input Format

The first line of input consists of an integer N, representing the number of gas stations.

The second line of input consists of N space-separated integers representing the amount of gas available at each station.

The third line of input consists of N space-separated integers representing the

cost of traveling from each station to the next.

Output Format

The output prints an integer representing the index of the starting gas station that allows you to complete the circuit, or -1 if no such station exists.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
3 4 5 1 2
Output: 3

Answer

```
// You are using Java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] gas = new int[n];
        int[] cost = new int[n];

        for (int i = 0; i < n; i++)
            gas[i] = sc.nextInt();

        for (int i = 0; i < n; i++)
            cost[i] = sc.nextInt();

        int total = 0, curr = 0, start = 0;

        for (int i = 0; i < n; i++) {
            int diff = gas[i] - cost[i];
            total += diff;
            curr += diff;
```

```
            if (curr < 0) {
                start = i + 1;
                curr = 0;
            }
        }

        if (total >= 0)
            System.out.print(start);
        else
            System.out.print(-1);
    }
}
```

Status : Correct ***Marks : 10/10***